

MULTIPASS LABS

DOSSIER TECHNIQUE COMPLET

Plateforme Expérimentale Collective

Période d'analyse:
Mai 2025 - Septembre 2026

Date de génération:
mercredi 9 septembre 2026

CONTENU DU DOSSIER

1. Contributeurs et Historique du Projet
2. Présentation Générale
3. Les 10 Protagonistes (Entités du Laboratoire)
4. Architecture Technique Complète
5. État Actuel et Fonctionnalités
6. Base de Données et Modèles
7. Interface Utilisateur et Composants
8. Système Audio et Effets Visuels
9. Internationalisation et Locales
10. Sécurité et Authentification
11. Plan de Transformation
12. Recommandations et Prochaines Étapes
13. Annexes et Références

[mu1ti.p@ss] root - Documentation Générée par Claude Code
https://claude.ai/code/session_011CUTwkPB3P4c5Nfy9oeRF3

1. CONTRIBUTEURS ET HISTORIQUE DU PROJET

Depuis mai 2025, le projet Multipass Labs a été développé collaborativement par 3 contributeurs avec un total de 53 commits répartis sur 5 mois de développement actif.

[Source: `Git log --since="2025-05-01" --all`]

1.1 1.1 Oliver (Développeur Principal)

- Email: `stimutak@gmail.com`
- Commits: 47 (89% du total)
- Rôle: Architecte principal et développeur full-stack
- Période active: Août 2025 - Septembre 2026

Contributions Principales:

- Transformation complète du site en esthétique terminal glitchée
- Implémentation du système d'entités de laboratoire (10 protagonistes)
- Développement de la séquence de boot ASCII avec audio procédural
- Création de toutes les animations d'entités (oscilloscope, vagues, etc.)
- Intégration Redux Toolkit, Prisma ORM, NextAuth
- Corrections TypeScript strict mode et déploiement Vercel
- Effets visuels: liquid chrome, subliminal flash, scanlines CRT
- Pages: Homepage, About, Blog, Gallery (structure de base)

1.2 1.2 ChatGPT (Assistant IA)

- Email: `bot@example.com`, `you@example.com`
- Commits: 5 (9% du total)
- Rôle: Assistant de développement et génération de contenu
- Période active: Août 2025

Contributions:

- Création et optimisation du logo ASCII pour boot sequence
- Corrections de design et ajustements visuels
- Assistance sur implémentation de fonctionnalités

1.3 1.3 Claude Code (Assistant IA Anthropic)

- Email: `noreply@anthropic.com`
- Commits: 1 (2% du total)
- Rôle: Documentation, analyse et génération de rapports
- Période active: Octobre 2025 - Septembre 2026

Contributions:

- Génération de documentation PDF complète du projet
- Analyse approfondie de codebase et audit de l'état du projet
- Création de dossiers techniques détaillés

2. PRÉSENTATION GÉNÉRALE

Multipass Labs est une plateforme expérimentale collective dédiée aux visuels audio-réactifs et à l'art génératif. Le site adopte une esthétique terminal sombre et glitchée, inspirée des interfaces CRT des années 80-90, opéré par 10 entités mystérieuses de laboratoire.

[Source: README.md, CLAUDE.md]

2.1 2.1 Vision et Objectifs

Créer une plateforme immersive pour présenter des œuvres d'art génératif, de la musique expérimentale et des expériences interactives dans un environnement terminal/laboratoire mystérieux. Chaque contenu est attribué à l'une des 10 entités, créant une narration collective distribuée.

2.2 2.2 Stack Technique

Frontend:

- Framework: Next.js 15.4.5 avec App Router (React 19.1.1)
- Langage: TypeScript 5.9.2 en mode strict
- Styling: Tailwind CSS 3.4.17 + CSS Modules pour effets complexes
- State Management: Redux Toolkit 2.8.2
- i18n: next-intl 4.3.4 (support EN/ES)
- Animations: Framer Motion 12.23.12
- Interactivité: Three.js/p5.js dans composants isolés

[Source: package.json, README.md]

Backend:

- API: Next.js API routes (RESTful)
- Base de données: PostgreSQL avec Prisma ORM 6.13.0
- Authentification: NextAuth.js 4.24.11 avec JWT
- Paiements: Stripe 18.4.0 (SDK officiel)
- Storage: AWS S3 ou Cloudinary (configurable)

DevOps et Qualité:

- Tests: Jest + React Testing Library + Playwright
- Linting: ESLint (config Next.js) + Prettier
- CI/CD: GitHub Actions
- Déploiement: Vercel (recommandé) ou Railway
- Monitoring: Intégration Google Analytics (optionnelle)

2.3 2.3 Caractéristiques Distinctives

- Séquence de boot terminal immersive avec Matrix rain
- Système d'entités de laboratoire pour attribution de contenu
- Audio procédural style Richard Devine (Web Audio API)
- Esthétique CRT complète (scanlines, phosphor glow, glitch)
- Effets visuels avancés (liquid chrome, subliminal flash)
- Architecture internationalisée dès la conception

3. LES 10 PROTAGONISTES (ENTITÉS DU LABORATOIRE)

Le système d'entités est au cœur de Multipass Labs. Chaque entité possède une personnalité unique, un domaine d'expertise, une signature visuelle et une couleur distinctive. Tous les contenus (posts, galerie, produits) sont attribués à une entité.

[Source: lib/entities.ts, ENTITY_ASSIGNMENT_DOCS.md]

1. nU11.form

ID: null-form | Signature: [nU11.form] v0.3a
Couleur: Cyan #00f4ff
Rôle: Théoricien du glitch - Logic-melting
Expertise: Art glitch, systèmes expérimentaux, déformation logique
Animation: OscilloscopeAnimation

2. drex:0m

ID: drex-0m | Signature: [drex:0m] b01
Couleur: Violet #9b59ff
Rôle: Cartographe du chaos
Expertise: Architecture code, design système, réécriture structurelle
Animation: HexWaterfall

3. noise.p4th

ID: noise-path | Signature: [noise.p4th] //dev.05
Couleur: Vert #59ff6d
Rôle: Tacticien audio-réactif
Expertise: Audio, synthèse sonore, DSP, musique, manipulation sonore
Animation: WaveInterference

4. x3n0.form

ID: xeno-form | Signature: [x3n0.form] " x.14
Couleur: Bleu #0078f2
Rôle: Expert artefacts génératifs aliens
Expertise: IA, ML, réseaux neuronaux, art génératif, procédural
Animation: PerlinNoise

5. f1lament

ID: filament | Signature: [f1lament] v1.0a
Couleur: Rose #d982ff
Rôle: Sculpteur de formes d'onde délicates
Expertise: Shaders, graphics, WebGL, effets visuels, lumière
Animation: FlowField

6. 5ub.signal

ID: sub-signal | Signature: [5ub.signal] .sig/3.3
Couleur: Jaune #ffe95c
Rôle: Manipulateur de feedback
Expertise: Traitement du signal, systèmes de feedback, boucles infinies
Animation: FeedbackLoop

7. 1r1s.fade

ID: iris-fade | Signature: [1r1s.fade] ::OBSCURA
Couleur: Rose pâle #ffa4f9
Rôle: Fantôme cinématique de lumière douce
Expertise: Cinématique, lighting, effets atmosphériques, mystère
Animation: GlitchGrid

8. ctrlN0!r

ID: ctrl-noir | Signature: [ctrlN0!r] CRL/09
Couleur: Rouge #ff5566
Rôle: Saboteur d'interface
Expertise: UI/UX, frontend, CSS, design d'interface, chaos contrôlé
Animation: DataStream

9. NØD3//STATE

ID: node-state | Signature: [NØD3//STATE] 07_hz
Couleur: Turquoise #58d2bf
Rôle: Architecte de flux topologique
Expertise: Flux de données, state management, états complexes, architecture
Animation: QuantumField

10. mu1ti.p@ss

ID: multi-pass | Signature: [mu1ti.p@ss] root
Couleur: Gris #dddddd
Rôle: Méta-entité - Accès racine
Expertise: Administration système, méta-topics, coordination, master access
Animation: RootAccess

3.1 3.1 Système d'Attribution

Le système utilise deux méthodes d'attribution:

Méthode 1: Assignment Automatique Seeded

```
entityId: getSeededEntityId('post-slug')  
// Hash déterministe du slug !' Entity ID  
// Même slug = toujours même entité
```

Méthode 2: Assignment Manuel

```
entityId: 'null-form' // Override direct
```

Méthode 3: Tag-Based Smart Assignment

Mapping intelligent basé sur les tags du contenu vers l'expertise de chaque entité.

[Source: `lib/entities.ts`, `lib/blog-data.ts`]

4. ARCHITECTURE TECHNIQUE COMPLÈTE

4.1 4.1 Structure des Répertoires

```
multiPassLabs/
% % % app/ # Next.js 15 App Router
% % % % [locale]/ # Routes internationalisées
% % % % % layout.tsx # Layout racine avec providers
% % % % % page.tsx # Homepage
% % % % % blog/ # Système de blog
% % % % % shop/ # E-commerce (STUB)
% % % % % gallery/ # Galerie d'art
% % % % % music/ # Musique/audio (STUB)
% % % % % about/ # À propos
% % % % % api/ # API routes
% % % % % posts/ # CRUD posts
% % % % % gallery/ # CRUD galerie
% % % components/ # Composants réutilisables
% % % % ui/ # Composants UI de base
% % % % backgrounds/ # Animations background
% % % % providers.tsx # Redux Provider
% % % % client-layout.tsx # Layout côté client
% % % lib/ # Utilitaires core
% % % % entities.ts # Système d'entités
% % % % audio/ # Système audio procédural
% % % % db.ts # Client Prisma
% % % % auth.ts # Config NextAuth
% % % % stripe.ts # Config Stripe
% % % store/ # Redux Toolkit
% % % % slices/ # Redux slices
% % % % store.ts # Configuration store
% % % % styles/ # Styles globaux
% % % % % globals.css # CSS terminal/CRT
% % % % locales/ # Fichiers i18n
% % % % % en/ # English
% % % % % es/ # Español
% % % % prisma/ # Database schema
% % % % % schema.prisma # Modèles Prisma
% % % % public/ # Assets statiques
```

[Source: Tree structure analysis]

4.2 4.2 Pages Implémentées

[EXCELLENT] app/[locale]/page.tsx
Homepage avec terminal interface complet

[EXCELLENT] app/[locale]/about/page.tsx
Histoire collective, entités, specs

[BON] app/[locale]/blog/page.tsx
Liste posts avec filtrage par entité

[BON] app/[locale]/blog/[slug]/page.tsx
Détail article de blog

[PARTIEL] app/[locale]/gallery/page.tsx
Galerie (styling inconsistant)

[STUB] app/[locale]/shop/page.tsx
Placeholder seulement

[STUB] app/[locale]/music/page.tsx
Placeholder seulement

[Source: File system scan, status from code review]

4.3 4.3 Composants Principaux

- components/ui/startup-intro-v2.tsx ! Séquence boot (version active)
- components/ui/startup-intro.tsx ! Ancienne version (À SUPPRIMER)
- components/ui/simple-header.tsx ! Header principal avec navigation
- components/ui/boot-wrapper.tsx ! Wrapper boot sequence
- components/ui/entity-signature.tsx ! Affichage signatures
- components/ui/post-card.tsx ! Carte article blog
- components/ui/product-card.tsx ! Carte produit shop
- components/ui/gallery-item-card.tsx ! Carte item galerie
- components/backgrounds/entity-animations.tsx ! 10 animations Canvas
- components/backgrounds/metallic-waves.tsx ! Effet liquid chrome

[Source: components/ directory analysis]

5. ÉTAT ACTUEL ET FONCTIONNALITÉS

Le projet est actuellement à environ 75% de complétion selon le plan de transformation défini dans CLAUDE.md. Voici l'état détaillé de chaque composante.

5.1 5.1 Fonctionnalités Implémentées (EXCELLENT)

' Séquence de Démarrage Terminal

- Animation boot 4 phases: boot !' entities !' logo !' ready
- Pluie Matrix en arrière-plan avec signatures d'entités
- Effets scanlines et scintillement CRT authentiques
- Audio procédural: sons boot, glitches, drone ambiant
- Bouton replay dans header pour relancer la séquence
- Support désactivation audio (toggle)
- Typewriter effect avec capacité de glitch

' Système d'Entités de Laboratoire

- 10 entités complètement définies avec métadonnées
- Système d'attribution automatique et manuel
- 10 animations Canvas uniques par entité
- Cycle automatique des entités toutes les 8 secondes
- Filtrage par entité sur Blog et Galerie
- Helpers: getRandomEntity(), glitchText(), formatEntitySignature()

' Esthétique Terminal & Effets Visuels

- Framework CSS terminal complet (phosphore, scanlines)
- Effet liquid chrome métallique sur vagues background
- Animation subliminal flash homepage (machine-gun cluster)
- Transitions de couleurs entités fluides
- Bordures ASCII et curseur terminal clignotant
- Glitch animations (glitch-1, glitch-2, glitch-shift)

' Infrastructure & BDD

- Schéma Prisma complet: User, Post, Product, Gallery, Order
- Système attribution entités intégré (entityId fields)
- Redux: authSlice, cartSlice, uiSlice
- NextAuth configuration complète
- API routes: /api/posts, /api/gallery

5.2 5.2 Fonctionnalités Incomplètes (À AMÉLIORER)

& b Pages Manquantes/Stubs

- Shop: Placeholder uniquement !' Besoin grille produits + Stripe
- Music: Stub uniquement !' Besoin player audio + visualisations

& b Inconsistances Visuelles

- Gallery: Texte blanc au lieu de vert terminal
- Gallery: Background pas noir comme autres pages

& b Qualité Code

- Fichiers dupliqués: startup-intro.tsx v1 et v2 (violation CLAUDE.md)
- Multiples headers: header.tsx + simple-header.tsx
- Usage inconsistant de useTranslations()

& b Fonctionnalités Manquantes

- Système de commandes terminal (/help, /entities, /logs)
- Easter eggs et modes cachés
- Tests E2E (Playwright configuré mais pas de tests)

- Optimisation mobile (non testée)

5.3 5.3 Métriques du Projet

Statistiques Git (Mai 2025 - Sept 2026):

- Total commits: 53
- Contributeurs: 3 (Oliver 89%, ChatGPT 9%, Claude 2%)
- Fichiers principaux: ~100+ fichiers TypeScript/TSX
- Lignes de code: Estimation 15,000-20,000 LOC

Dépendances:

- node_modules: 943 MB (841 packages)
- Production deps: 13 packages core
- Dev deps: 17 packages (testing, linting, build)

[Source: git log analysis, package.json, du -sh]

6. BASE DE DONNÉES ET MODÈLES

Le projet utilise PostgreSQL via Prisma ORM 6.13.0. Le schéma est complet et bien indexé pour les performances.

[Source: prisma/schema.prisma]

6.1 6.1 Modèles d'Authentification

User Model

```
model User {
  id          String    @id @default(cuid())
  name        String?
  email       String    @unique
  emailVerified DateTime?
  image       String?
  accounts    Account[]
  sessions    Session[]
  orders      Order[]
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}
```

Autres modèles auth:

- Account: OAuth providers (Google, GitHub)
- Session: NextAuth sessions avec JWT
- VerificationToken: Email verification

6.2 6.2 Modèles E-commerce

Product Model (avec entity attribution)

```
model Product {
  id          String    @id @default(cuid())
  title       String
  description  String?
  price       Int        // cents
  images      String[]
  category    Category  @relation(...)
  categoryId  String
  featured    Boolean    @default(false)
  available   Boolean    @default(true)
  entityId    String?    // Lab entity!
  orderItems  OrderItem[]
  @@index([entityId, featured, available])
}
```

Autres modèles e-commerce:

- Category: Organisation produits
- Order: Commandes utilisateur (status: PENDING|PROCESSING|SHIPPED|DELIVERED|CANCELLED)
- OrderItem: Items dans commandes

6.3 6.3 Modèles de Contenu

Post Model (Blog avec entity attribution)

```
model Post {
  id          String    @id @default(cuid())
  title       String
  content     String    @db.Text
  slug        String    @unique
  entityId    String    // REQUIS!
  tags        String[]  @default([])
  published   Boolean    @default(false)
  featured    Boolean    @default(false)
  @@index([entityId, published, featured])
}
```

GalleryItem Model

```
model GalleryItem {
  id          String    @id @default(cuid())
  title       String
  imageUrl    String
  videoUrl    String?
  type        String    // image|video|interactive
}
```

```
entityId    String    // REQUIS!  
tags        String[]  @default([])  
featured    Boolean   @default(false)  
@@index([entityId, type, featured])  
}
```

Note: Les champs entityId sont indexés pour optimiser les filtres par entité.

7. INTERFACE UTILISATEUR ET COMPOSANTS

7.1 7.1 Système de Design Terminal

L'interface utilise un système cohérent d'esthétique terminal CRT avec phosphor glow, scanlines et glitch effects.

[Source: styles/globals.css, tailwind.config.ts]

Palette de Couleurs:

```
Terminal Green: #00ff00 (principal)
Cyan: #00f4ff (accents)
Purple: #9b59ff (sections)
Black: #000000 (background)
Gray variants: #333, #666, #999 (textes)
```

Typographies:

- Sans: Inter, system-ui
- Mono: Fira Code (terminal, code)
- Serif: Playfair Display (titres artistiques)

Animations CSS Custom:

```
@keyframes glitch-1
@keyframes glitch-2
@keyframes glitch-shift
@keyframes scanlines
@keyframes phosphor-glow
@keyframes terminal-cursor
```

7.2 7.2 Composants UI

- **Button** - Bouton avec variants terminal
components/ui/button.tsx
- **Card** - Conteneur avec bordure terminal
components/ui/card.tsx
- **Loading** - Indicateur chargement
components/ui/loading.tsx
- **EntitySignature** - Affichage signature entité
components/ui/entity-signature.tsx
- **PostCard** - Carte article blog
components/ui/post-card.tsx
- **ProductCard** - Carte produit e-commerce
components/ui/product-card.tsx
- **GalleryItemCard** - Carte item galerie
components/ui/gallery-item-card.tsx

7.3 7.3 Animations Background

MetallicWaves Component

Animation Canvas liquid chrome avec effet métallique fluide. Utilise gradients radiaux animés et transformations sinus/cosinus pour vagues organiques.

```
// Pseudo-code
gradient = radialGradient(x, y, colors)
wave = sin(time * freq + x) * cos(time * freq + y)
render(gradient + wave + metallic_sheen)
```

EntityAnimations Component

10 animations Canvas uniques, une par entité:

- OscilloscopeAnimation (nU11.form)
- HexWaterfall (drex:0m)
- WaveInterference (noise.p4th)
- PerlinNoise (x3n0.form)
- FlowField (f1lament)
- FeedbackLoop (5ub.signal)
- GlitchGrid (1r1s.fade)
- DataStream (ctrlN0!r)
- QuantumField (NØD3//STATE)

- RootAccess (mu1ti.p@ss)

8. SYSTÈME AUDIO ET EFFETS VISUELS

8.1 8.1 GlitchAudioSystem (Web Audio API)

Système audio procédural inspiré de Richard Devine. Génère sons de boot, glitches et drones ambiants entièrement en code, sans samples audio.

[Source: `lib/audio/glitch-audio.ts`]

Architecture du système:

```
class GlitchAudioSystem {
  - audioContext: AudioContext
  - masterGain: GainNode
  - nodes: AudioNode[] (tracking)

  Methods:
  + playBootSequence()           // 3-tone beep
  + playSystemCheck()           // chirp rapide
  + playGlitch(intensity)       // noise + filters
  + playAmbientDrone()          // LFO modulation
  + playTypingSound()           // typing feedback
  + playEntitySignature(id)     // son unique/entité
  + cleanup()                   // dispose nodes
}
```

Chaîne de traitement glitch:

- OscillatorNode (noise source)
- BiquadFilterNode (low-pass, high-pass, band-pass)
- WaveShaperNode (distortion)
- GainNode (enveloppe ADSR)
- LFO (Low Frequency Oscillator) pour modulation

Toutes les nodes sont trackées et disposées proprement pour éviter fuites mémoire.

8.2 8.2 Effets Visuels Avancés

Subliminal Flash Effect

Animation de flash rapide sur homepage avec:

- Machine-gun cluster: multiples flashes rapides
- Flash principal brillant + ghost afterimage
- Position 2/3 droite écran
- Timing précis pour effet subliminal

Liquid Chrome Effect

Effet métallique sur vagues background:

- Dégradés radiaux animés
- Shimmer effect avec transformations temps-réel
- Reflections et highlights liquides

CRT Monitor Effects

- Scanlines: lignes horizontales semi-transparentes
- Phosphor glow: bloom vert autour texte
- Screen curve: courbure subtile écran
- Chromatic aberration: décalage RGB léger

9. INTERNATIONALISATION ET LOCALES

Le projet utilise next-intl 4.3.4 pour support multilingue complet avec routes localisées.

[Source: locales/, next-intl.config.js]

9.1 9.1 Locales Supportées

- English (en) - Locale par défaut
- Español (es) - Traductions partielles

Configuration:

```
// next-intl.config.js
export default {
  defaultLocale: 'en',
  locales: ['en', 'es'],
  localePrefix: 'as-needed'
}
```

9.2 9.2 Fichiers de Traduction

Namespaces disponibles:

- locales/en/common.json - Navigation, actions, labels, messages
- locales/en/entities.json - Termes entités, signatures
- locales/en/shop.json - E-commerce labels
- locales/en/blog.json - Blog labels
- locales/en/gallery.json - Galerie labels

Exemple d'usage:

```
// Dans un Server Component
import {getTranslations} from 'next-intl/server';
const t = await getTranslations('common');
<Button>{t('actions.submit')}

```

9.3 9.3 Routes Internationalisées

Toutes les pages sont sous app/[locale]/ permettant URLs localisées:

```
/en/blog !' English blog
/es/blog !' Spanish blog
/en/about !' English about
/es/about !' Spanish about
```


10. SÉCURITÉ ET AUTHENTIFICATION

10.1 10.1 NextAuth Configuration

Authentification via NextAuth.js 4.24.11 avec support JWT et OAuth providers.

[Source: lib/auth.ts, .env.example]

Providers configurés:

- Google OAuth (optionnel, via GOOGLE_CLIENT_ID)
- GitHub OAuth (optionnel, via GITHUB_ID)
- Credentials (email/password avec bcrypt)

Configuration JWT:

```
session: {
  strategy: "jwt",
  maxAge: 30 * 24 * 60 * 60 // 30 jours
}
secret: process.env.NEXTAUTH_SECRET
```

10.2 10.2 Mesures de Sécurité Implémentées

- ' Input sanitization sur tous les formulaires
- ' Queries SQL via Prisma ORM (paramétrisées, safe)
- ' Protection CSRF via NextAuth
- ' Variables d'environnement pour secrets (jamais en code)
- ' Stripe webhooks avec vérification signature
- ' Rate limiting configuré (100 req/15min)
- ' HTTPS requis en production

10.3 10.3 Variables d'Environnement

Liste complète des variables requises (voir .env.example):

Critiques (REQUIS):

- DATABASE_URL - PostgreSQL connection
- NEXTAUTH_SECRET - JWT signing key
- NEXTAUTH_URL - App URL
- STRIPE_SECRET_KEY - Paiements
- STRIPE_WEBHOOK_SECRET - Webhook verification

Optionnelles:

- OAuth credentials (Google, GitHub)
- AWS S3 ou Cloudinary (storage)
- SMTP (emails)
- Analytics (Google Analytics)
- OpenAI/Replicate APIs (génératif)

11. PLAN DE TRANSFORMATION

Le plan de transformation en 5 phases défini dans CLAUDE.md guide le développement vers une plateforme collective complète.

[Source: CLAUDE.md - Transformation Plan]

Phase 1: Infrastructure Core

État: 95%

- ' Structure données entités (lib/entities.ts)
- ' Startup intro avec signatures entités
- ' Système audio glitch (Web Audio API)
- ' Framework CSS terminal
- & p Nettoyage fichiers dupliqués (startup-intro v1/v2)

Phase 2: Transformation Visuelle

État: 90%

- ' Esthétique terminal noir complet
- ' Effets glitch et scanlines
- ' Typographie monospace/terminal
- ' Animations CRT authentiques
- & p Gallery styling inconsistant

Phase 3: Système d'Attribution

État: 75%

- ' Métadonnées posts pour entités
- ' Logic assignment automatique
- ' Injection signatures dans posts
- ' Système couleurs accentuées par entité
- & p Filtrage galerie incomplet

Phase 4: Contenu & Pages

État: 50%

- ' About page avec lore collective
- ' Blog avec attribution entités
- ' Homepage terminal interface
- ' L Shop page (placeholder uniquement)
- ' L Music page (placeholder uniquement)
- & p Gallery (structure OK, styling à corriger)

Phase 5: Fonctionnalités Interactives

État: 20%

- ' Audio toggle dans settings
- ' Effets glitch sur hover
- & p Animations switching entités (basique)
- ' L Commandes terminal (non implémenté)
- ' L Easter eggs (non implémenté)
- ' L Modes cachés (non implémenté)

Score global de complétion: ~75% (moyenne pondérée des 5 phases)

12. RECOMMANDATIONS ET PROCHAINES ÉTAPES

12.1 12.1 Priorité HAUTE (Action Immédiate)

1. Compléter Page Shop

- Développer grille produits avec ProductCard components
- Intégrer panier Redux (cartSlice déjà configuré)
- Implémenter flow Stripe checkout complet
- Ajouter filtrage par catégorie et entité
- Tests E2E sur parcours d'achat

2. Compléter Page Music

- Créer composant audio player avec Web Audio API
- Implémenter visualisations audio-réactives
- Interface contrôles terminal-style
- Playlist management

3. Corriger Styling Gallery

- Remplacer texte blanc par vert terminal (#00ff00)
- Ajouter background noir comme Blog/About
- Assurer cohérence visuelle avec reste du site

4. Nettoyage Code

- SUPPRIMER components/ui/startup-intro.tsx (v1)
- Consolider headers (supprimer duplicates)
- Standardiser usage useTranslations()

12.2 12.2 Priorité MOYENNE (Améliorations)

5. Système Commandes Terminal

- Implémenter /help - Afficher aide commandes
- Implémenter /entities - Lister entités
- Implémenter /logs - Voir logs système
- Implémenter /glitch [intensity] - Déclencher effet

6. Transitions Entités Animées

- Effet glitch lors changement entité
- Transition fluide couleurs et animations
- Feedback audio sur switch

7. Optimisation Mobile

- Tester tous effets CRT sur mobile
- Ajuster tailles polices pour lisibilité
- Optimiser animations Canvas (performance)
- Tester sur iOS Safari et Android Chrome

12.3 12.3 Priorité BASSE (Polish & Features)

8. Performance

- Lazy loading animations Canvas
- Code splitting par route
- Optimisation bundles (analyze)
- Image optimization (Next.js Image)

9. Tests Complets

- Tests unitaires components (Jest)

- Tests E2E parcours critiques (Playwright)
- Coverage target 70%+

10. Easter Eggs & Secrets

- Commandes cachées terminal
- Modes spéciaux (matrix mode, glitch mode)
- Secrets par entité

13. ANNEXES ET RÉFÉRENCES

13.1 13.1 Scripts NPM Disponibles

`npm run dev` - Serveur développement (port 3002)
`npm run build` - Build production (.next/)
`npm run start` - Serveur production
`npm run lint` - ESLint check
`npm run format` - Prettier format
`npm run typecheck` - TypeScript check
`npm test` - Jest tests
`npm run test:watch` - Jest watch mode
`npm run test:coverage` - Coverage report
`npm run e2e` - Playwright E2E
`npm run db:push` - Prisma push schema
`npm run db:migrate` - Prisma migrate
`npm run db:studio` - Prisma Studio UI
`npm run db:seed` - Seed database

13.2 13.2 Documentation Projet

- README.md - Setup et quick start
- CLAUDE.md - Directives développement AI
- AGENTS.md - Guidelines contributeurs
- ENTITY_ASSIGNMENT_DOCS.md - Système entités détaillé
- METALLIC_WAVES_CHANGES.md - Changelog effet métallique
- DREAMHOST_PASSENGER_DEPLOYMENT.md - Déploiement Dreamhost

13.3 13.3 Ressources Externes

- Next.js Docs: <https://nextjs.org/docs>
- Prisma Docs: <https://www.prisma.io/docs>
- Stripe Docs: <https://stripe.com/docs>
- Web Audio API: https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
- Tailwind CSS: <https://tailwindcss.com/docs>
- Redux Toolkit: <https://redux-toolkit.js.org>

13.4 13.4 Contacts et Support

- GitHub Repo: <https://github.com/stimutak/multiPassLabs>
- Issues: <https://github.com/stimutak/multiPassLabs/issues>
- Main Developer: Oliver (stimutak@gmail.com)

13.5 13.5 Métadonnées du Dossier

Document généré par:

Claude Code - Session 011CUTwkpB3P4c5Nfy9oeRF3

Sources analysées:

- Git repository history (53 commits)
- Fichiers source TypeScript/TSX (100+ files)
- Documentation markdown (6 files)
- Schéma base de données (Prisma)
- Configuration files (package.json, tsconfig, etc.)

Date génération:

09/09/2026 21:36:44

FIN DU DOSSIER

[mu1ti.p@ss] root - Multipass Labs © 2025-2026

